

Play Store Analytics Dashboard

An Interactive, Time-Gated Data Visualization Project

Project Name:	Intership_Project
Your Name:	Shruti Mekala
College:	KC College
Internship Name:	Intership_Project
Date:	20/07/2026

Submitted as part of the internship curriculum

1. Introduction

This project is a multi-chart, web-based analytics dashboard built on top of the Google Play Store dataset. The goal of the project was to explore a large, real-world app-store dataset and present meaningful, filtered insights through a set of six distinct visualizations covering ratings, reviews, installs, revenue, category trends, and app characteristics such as size and sentiment.

Rather than presenting a static set of charts, the dashboard was designed to feel like a living product: each visualization is generated from a cleaned dataset using Python, exported as JSON, and rendered client-side using Plotly.js inside a styled HTML/CSS dashboard. A distinguishing feature of this project is that each chart is only visible during a specific window of the day (IST), simulating a scheduled reporting system where different insights become relevant at different times.

2. Objective

The objective of this project was to design and build an interactive analytics dashboard that:

- Cleans and processes a real-world, messy dataset (Google Play Store apps and user reviews). ● Applies meaningful business-style filters (rating thresholds, size limits, install counts, category rules) to surface useful insights rather than raw data dumps.
- Visualizes the data using a variety of chart types suited to the underlying question being asked (comparison, distribution, trend, geography, correlation).
- Presents the dashboard as a polished, responsive, front-end web page rather than a static notebook.

- Introduces a time-based visibility feature so each chart is only shown during its designated IST time window.

3. Technologies Used

Technology	Purpose
Python	Data cleaning, filtering, and aggregation logic
Pandas	Reading, cleaning, and transforming the CSV datasets
Plotly / Plotly.js	Rendering interactive charts in the browser
HTML	Structuring the dashboard layout and sections
CSS	Styling the dark-themed, responsive dashboard UI
JavaScript	Handling chart rendering, time-based visibility logic, and dropdown
GitHub	Version control and project hosting

4. Dataset

The project uses the publicly available Google Play Store dataset, split across two CSV files:

File	Description	Rows	Columns
Play_Store_Data.csv	App-level metadata: category, rating, size, installs, type, price, content rating, and last-updated date for ~10,000+ apps.	10,842	13
User_Reviews.csv	User review text and pre-computed sentiment polarity/subjectivity scores per app.	64,296	5

Note: row/column counts above reflect the standard public release of this dataset. If your working copy of Play_Store_Data.csv / User_Reviews.csv differs (e.g. after de-duplication), update these figures accordingly.

Together, these two files allowed the project to combine app-level metrics (installs, ratings, revenue) with review-level sentiment signals, which is specifically used to filter the bubble chart to apps that have genuine, subjective user-review content.

5. Data Cleaning

Raw Play Store data is notoriously messy — installs are stored as text with commas and plus signs, sizes are mixed units, and dates come as free text. Before any chart could be built, the following cleaning steps were applied:

- Removed null values — rows missing critical fields (rating, category, or installs) were dropped so aggregate statistics like average rating wouldn't be skewed.
- Converted installs to numbers — fields like "10,000+" were stripped of commas and plus signs and cast to integers so they could be summed, ranked, and compared numerically.

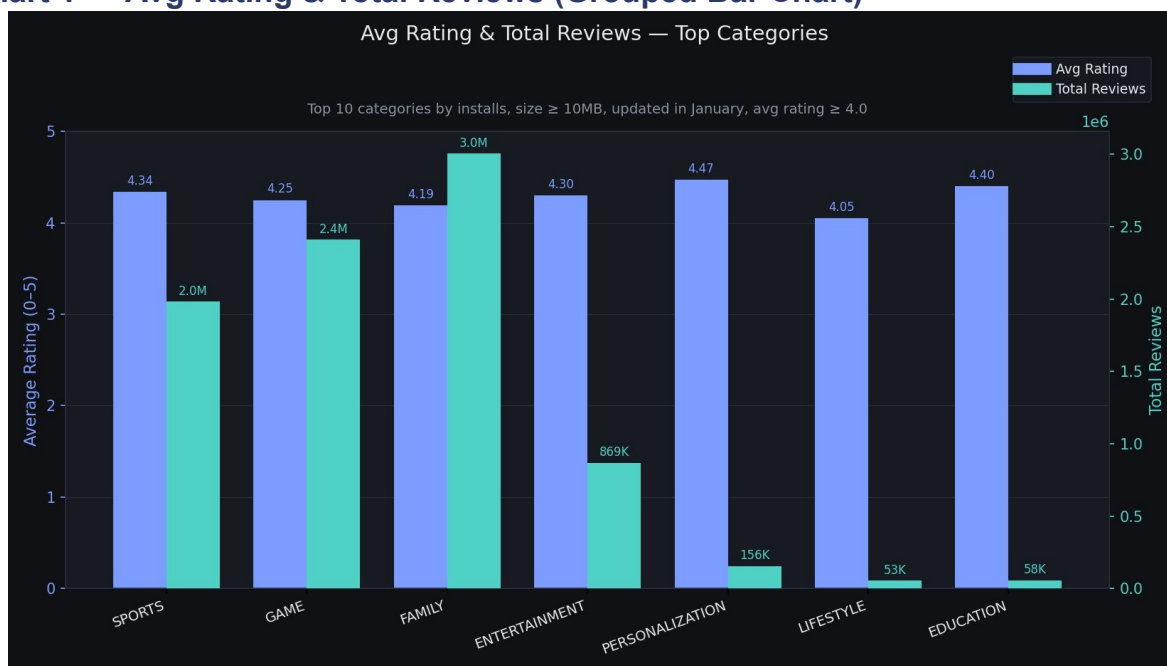
- **Converted dates** — the "Last Updated" text field was parsed into proper datetime objects, from which year-month and month-of-year values were derived for trend and monthly-filter charts.
- **Filtered unwanted rows** — records that didn't meet chart-specific business rules (e.g. size below a minimum, apps starting with excluded letters, or unrealistic values) were excluded before aggregation, keeping each chart focused on a clean, comparable subset of the data.

This cleaning step was performed once in Python (using Pandas) and the cleaned results were exported as small JSON files — one per chart — which the dashboard then loads directly, keeping the front end lightweight and free of any data-wrangling logic.

6. Visualizations

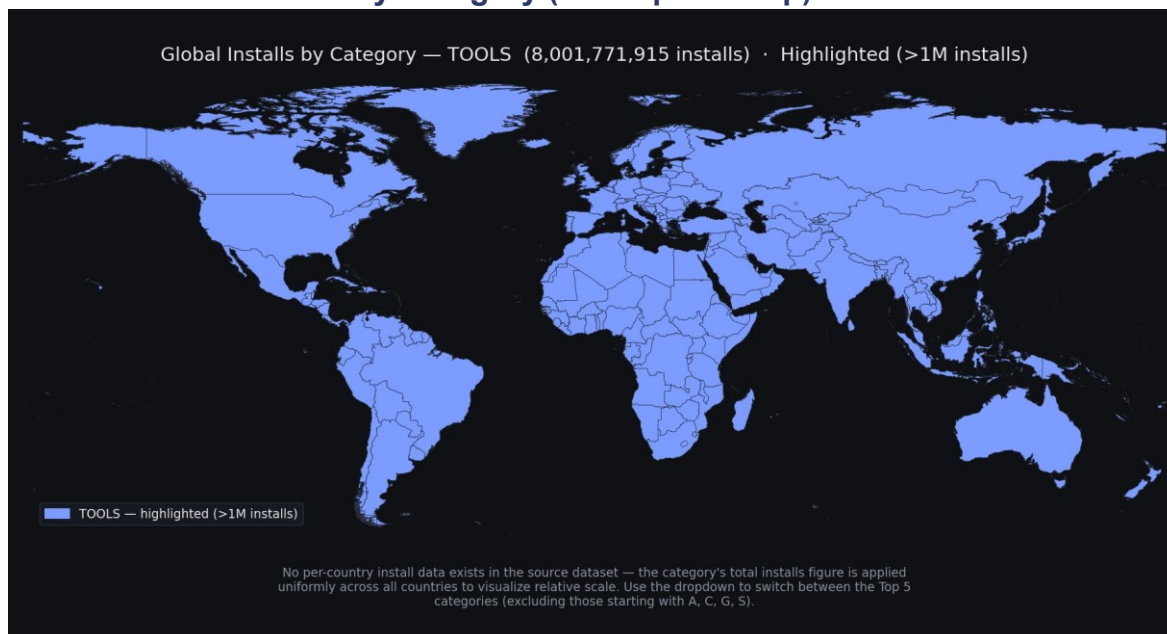
Six distinct charts were built, each answering a different analytical question about the dataset. Each chart is also gated to a specific time window (IST) in the live dashboard, so it is only visible to users during that period of the day.

Chart 1 — Avg Rating & Total Reviews (Grouped Bar Chart)



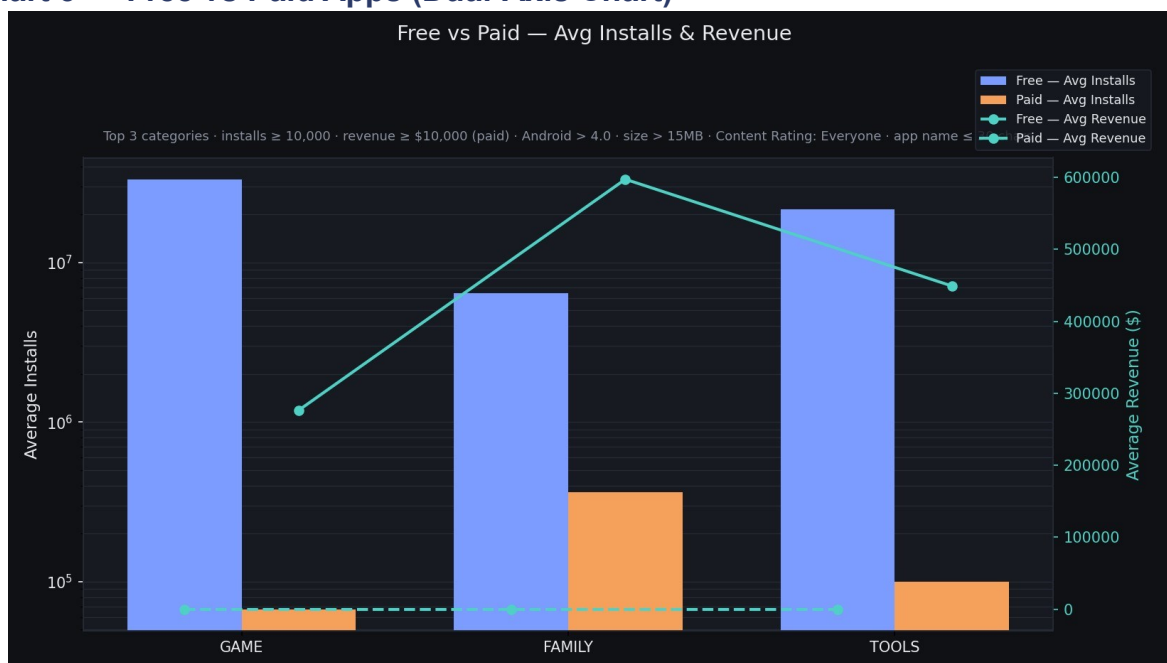
This grouped bar chart compares the average rating and total review count across the top app categories by installs (filtered to apps ≥ 10MB, last updated in January, with an average category rating of at least 4.0). Ratings are plotted on the left axis (0–5 scale) and total reviews on the right axis, letting viewers spot categories that are both well-reviewed and highly rated, such as Personalization and Education. Visible 3–5 PM IST.

Chart 2 — Global Installs by Category (Choropleth Map)



A world choropleth map visualizing the relative install scale of the top 5 categories (excluding categories starting with A, C, G, or S). Since the dataset has no per-country install breakdown, each category's total install figure is applied uniformly across all countries purely to communicate scale — categories exceeding 1 million installs are flagged as "highlighted." A dropdown in the live dashboard lets users switch between categories. Visible 6–8 PM IST.

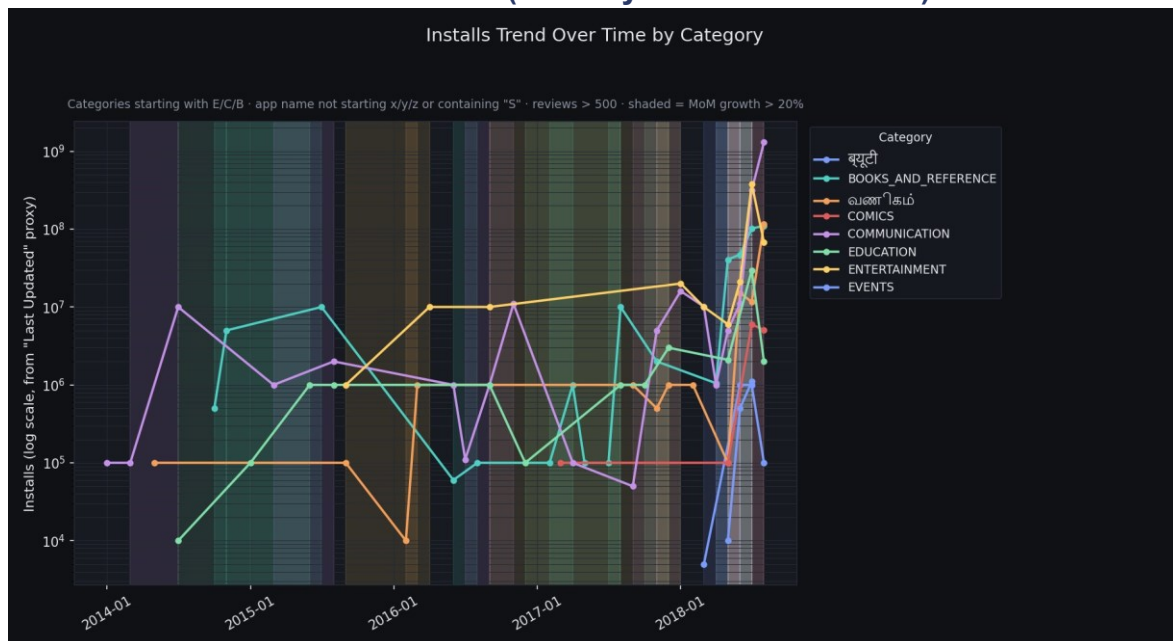
Chart 3 — Free vs Paid Apps (Dual-Axis Chart)



This dual-axis chart compares Free vs Paid apps across the top 3 categories by installs, using bars for average installs (log scale, left axis) and a line for average revenue (right axis). It clearly shows the expected trade-off: free apps dominate on install volume, while paid apps generate measurable

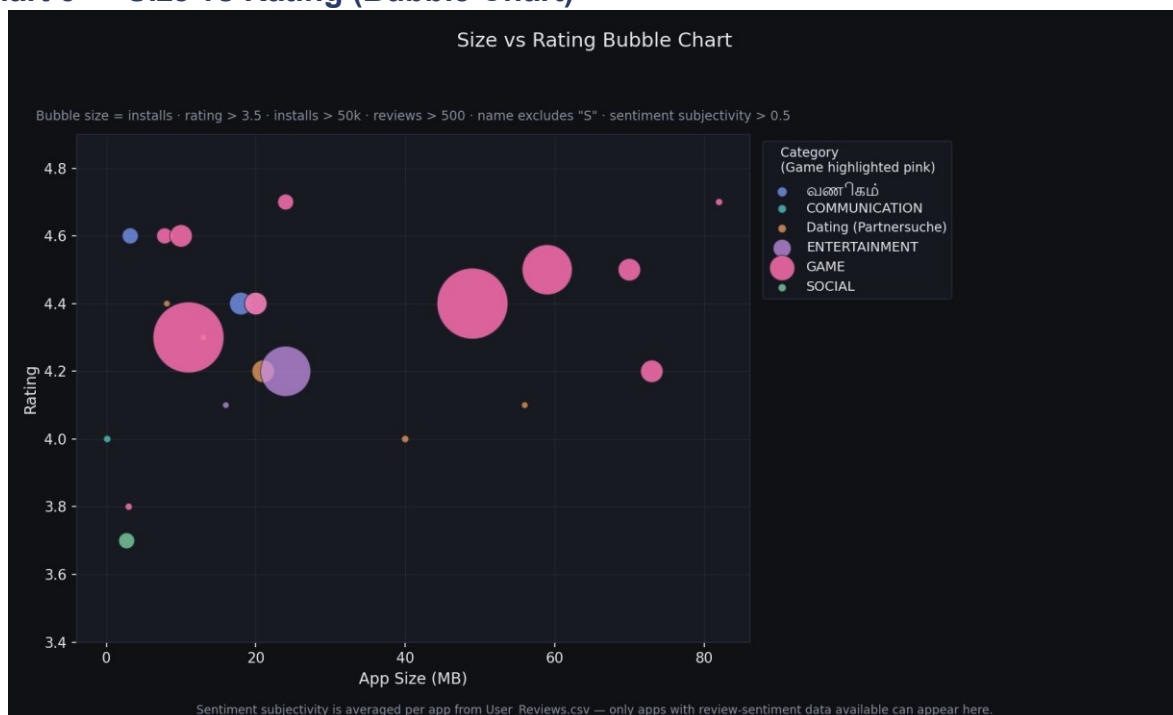
revenue despite far fewer installs. Filters applied include installs $\geq 10,000$, revenue $\geq \$10,000$ for paid apps, Android version > 4.0 , size $> 15\text{MB}$, and Content Rating = Everyone. Visible 1–2 PM IST.

Chart 4 — Installs Trend Over Time (Monthly Trend / Line Chart)



A multi-line time series showing installs over time (using each app's "Last Updated" date as a practical proxy for install activity, since the dataset has no true historical install log) for categories starting with E, C, or B. Shaded vertical bands mark months where a category's month-over-month growth exceeded 20%, making growth spikes easy to spot at a glance. Visible 6–9 PM IST.

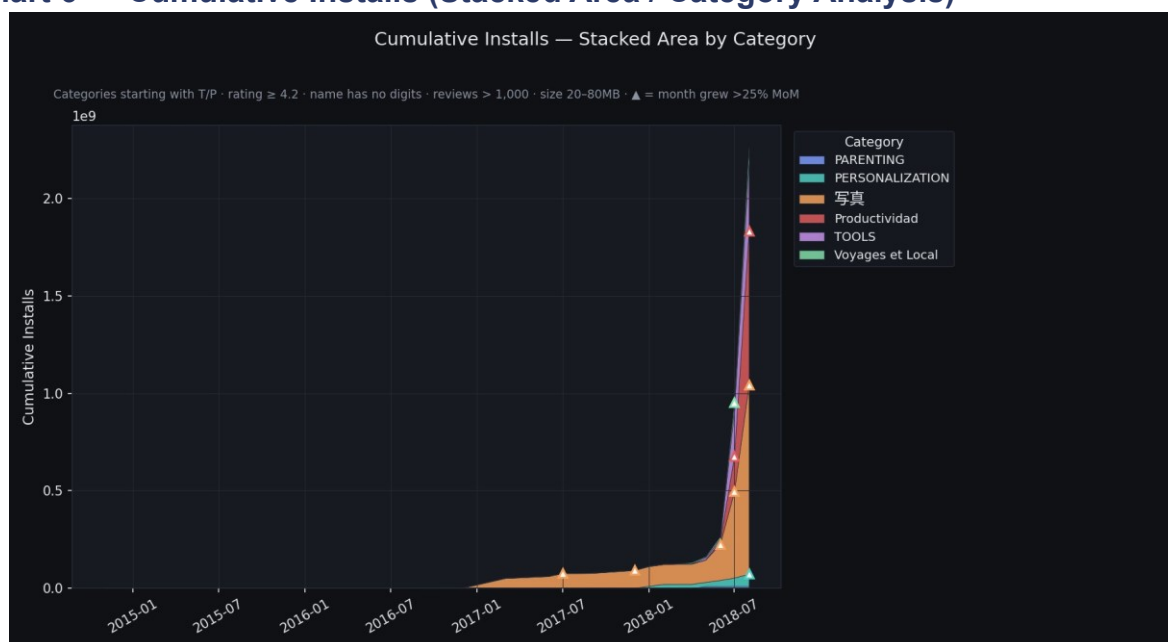
Chart 5 — Size vs Rating (Bubble Chart)



A bubble chart plotting app size (MB) against rating, where bubble size represents install count. This chart is restricted to apps with real review-sentiment data (subjectivity score >0.5 , sourced from User_Reviews.csv), rating >3.5 , installs $>50k$, and reviews >500 . The Game category is deliberately highlighted in pink to make it stand out, since it dominates the bubble sizes across most size ranges.

Visible 5–7 PM IST.

Chart 6 — Cumulative Installs (Stacked Area / Category Analysis)



A stacked area chart showing cumulative installs over time for categories starting with T or P (Travel & Local, Productivity, Photography, Tools, Parenting, Personalization), filtered to highly rated (≥ 4.2), well-reviewed ($>1,000$ reviews) apps between 20–80MB in size. Triangle markers flag months where a category's month-over-month installs grew more than 25%, most visibly in the sharp 2018 growth spike for Photography and Productivity. Visible 4–6 PM IST.

7. Features

Interactive Charts

All six visualizations are rendered with Plotly.js, giving users hover tooltips, zoom/pan controls, and legend-based series toggling directly in the browser — far more exploratory than a static image.

Time-Based Visibility

Each chart section is bound to a specific IST time window (e.g. 3–5 PM for Chart 1, 6–8 PM for Chart 2). Outside its window, the chart is hidden and replaced with a friendly "check back later" message, simulating a scheduled-reporting experience.

Filters

Every chart is backed by a deliberately chosen set of business-style filters (rating thresholds, size ranges, install/review minimums, category rules, and naming exclusions) so the visualizations highlight

meaningful subsets of the data rather than the entire noisy dataset. Chart 2 also includes a live category-selection dropdown.

Responsive Dashboard

The HTML/CSS layout uses a card-based design with a dark theme, flexible widths, and consistent spacing so the dashboard remains readable across different screen sizes, from desktop monitors to smaller laptop screens.

8. Conclusion

Working on this project gave me hands-on experience with the full lifecycle of a data visualization product — starting from a raw, imperfect CSV dataset all the way to a polished, interactive dashboard. On the data side, I learned how much real-world data cleaning matters: handling null values, converting text-based install counts and dates into usable numeric/datetime formats, and applying precise, chart-specific filters using Pandas.

On the visualization side, I learned to match each chart type to the question it needed to answer — using bar charts for direct comparison, a choropleth for geographic scale, a dual-axis chart for trade-off analysis, line and stacked-area charts for trends over time, and a bubble chart for multi-variable correlation. I also gained practical experience integrating Python-generated JSON data with a Plotly.js and HTML/CSS front end, and implementing a time-based visibility feature in JavaScript, which required thinking carefully about state, scheduling, and user experience beyond just the charts themselves.

Overall, this internship project strengthened my understanding of the end-to-end analytics workflow — data cleaning, aggregation, visualization design, and front-end presentation — and gave me confidence in building data products that are both technically sound and genuinely usable.